

Invoice Print System — Complete Developer Guide

This document explains how the **invoice print / PDF system** was built, how the pieces combine, and — most importantly — **exactly which file to edit** for every common kind of change (adding a column, changing colors, adding a field, adjusting the page, etc.).

Read this before touching any print-related code.

Table of Contents

1. What was built
 2. The three phases: Create → Show → Convert to PDF
 3. Component map (what each file does)
 4. ★ The "which file do I edit?" quick reference
 5. How the design fits together (architecture)
 6. Recipes — common modifications
 7. Where the data comes from (field names)
 8. How PDF generation works (the important part)
 9. How browser printing works
 10. Gotchas & troubleshooting
-

1. What was built

A **modern, portrait, one-page, print-ready invoice** that replaces the old dot-matrix style from the legacy "Apollo Tire & Wheel" software. It is fully integrated into the existing Next.js invoice module:

- **Any invoice** in the list can be opened on a dedicated print page.
- From that page the user can **browser-print** (Ctrl+P) or **download a PDF**.
- After **saving** an invoice (new or edit), the user is **prompted** with "Yes, View & Print" / "No, Go to List".

There are **no new backend changes** — it reads the same `/api/InvoiceMaster/{id}` data the rest of the app uses.

2. The three phases: Create → Show → Convert to PDF

PHASE 1 – CREATE

Invoice Form saves

- └ Edit flow → POST /api/InvoiceMaster/EditInvoice?id={id} (id from URL)
- └ Create flow → POST /api/InvoiceMaster/CreateInvoice (returns the saved invoice;
id read from response.invoiceMasterDto.id)

↓

Success dialog appears ("Invoice Saved Successfully")

- └ "No, Go to List" → /react-tables/transaction/invoice
- └ "Yes, View & Print" → /react-tables/transaction/invoice/{id}/print

PHASE 2 – SHOW

print page (server) → <InvoicePrintView invoiceId={id}/>

- client fetches GET /api/InvoiceMaster/{id} via SWR
- renders the invoice as styled HTML (the report design)

PHASE 3 – CONVERT

- └ Browser print : window.print() → CSS @media print rules take over
- └ Download PDF : html-to-image (DOM → PNG) → jsPDF (PNG → PDF file)

Phase 1 details (the save + prompt)

File: `src/app/components/react-tables/transaction/invoice-form/index.tsx`

After a successful save, the code:

1. Extracts the new invoice's primary key (`resultId`).
- **Edit:** `resultId = Number(invoiceId)` — straight from the URL param.
- **Create:** reads it from the API response. The endpoint returns the **full invoice object** (no `{status,msg,data}` wrapper), so the id is at `result.invoiceMasterDto.id`. The code tries several shapes defensively:

```
resultId =  
  r?.invoiceMasterDto?.id ??      // ← actual response shape  
  r?.data?.invoiceMasterDto?.id ??  
  r?.data?.id ??  
  r?.id ??  
  null
```

If nothing matches it logs a `console.warn` with the real response so you can fix the path.

1. Stores it in state: `setSavedInvoiceId(resultId)`.
2. Opens the dialog: `setSaveSuccessDialogOpen(true)`.
3. The dialog's **"Yes, View & Print"** button navigates to `/react-tables/transaction/invoice/{savedInvoiceId}/print`.

The dialog JSX lives at the **bottom** of the form's return statement (search for `Post-save success dialog`).

Phase 2 details (the print page)

Route file: `src/app/(DashboardLayout)/react-tables/transaction/invoice/[id]/print/page.tsx`

- This is a **server component** — it only supplies `metadata` (page title), a breadcrumb, and the id.
- It renders `<InvoicePrintView invoiceId={id} />`.
- The path convention matches the edit page: `.../invoice/[id]/edit` and `.../invoice/[id]/print` sit side by side.

Entry points to this page:

Where	File	Line
Invoice list dropdown → "View / Print"	<code>src/app/components/react-tables/transaction/invoice-datatable/index.tsx</code>	~350
Post-save dialog → "Yes, View & Print"	<code>src/app/components/react-tables/transaction/invoice-form/index.tsx</code>	~1320

Phase 3 details

- **Print button** → `window.print()` . Simple, and the CSS in `print.css` makes everything else happen (see §9).
- **Download PDF button** → clone-in-wrapper capture → PNG → jsPDF (see §8).

3. Component map

Folder: `src/app/components/react-tables/transaction/invoice-print/`

```
invoice-print/
├─ index.tsx           ← Main orchestrator. Fetches data, holds the action bar
                        (Print / Download PDF), arranges all sections, owns the
                        printRef for PDF capture.
├─ InvoiceHeader.tsx    ← Company name, address, phone + blue accent bar (logo spot)
├─ InvoiceMeta.tsx      ← Invoice #, Date, Payment Status badge
├─ CustomerInfo.tsx     ← Customer info (left) + Vehicle info (right), 2-col grid
├─ LineItemsTable.tsx  ← ★ Line-items table (Tax/Qty/Description/Size/Bolt/Price/Amount)
                        + inline financial summary (SubTotal, Tax, Labour, Discount, Total)
├─ PaymentHistory.tsx  ← Payment records table (# / Amount / Date / Type) – hidden if none
├─ FinancialSummary.tsx ← Sidebar summary (SubTotal, Tax, Labour, Discount, Adjustment,
                        Total, Adjusted Total, Amount Paid, Balance Due)
├─ WheelIndicator.tsx  ← Read-only car silhouette + LF/RF/LR/RR wheel badges
├─ InvoiceFooter.tsx    ← Notes, legal terms, signature line, store name
└─ print.css           ← All @media print rules (page size, hide chrome, fonts)
```

How they combine — the tree in `index.tsx` :

```

<InvoicePrintView invoiceId={id}> // index.tsx
|   └─ SWR: data = GET /api/InvoiceMaster/{id}
|   └─ → const master   = data.invoiceMasterDto
|   └─ → const details  = data.invoiceDetailsDto ?? []
|   └─ → const payments = data.invoicePaymentsDto ?? []
|
|   [Action bar – hidden when printing] → Print | Download PDF
|
|   <div ref={printRef} class="invoice-print-root"> ← THE report
|   |   └─ <InvoiceHeader master={master} /> ← 1. Company header
|   |   └─ <InvoiceMeta  master={master} /> ← 2. Invoice #, date, status
|   |   └─ <CustomerInfo master={master} /> ← 3. Customer + vehicle
|   |   └─ <LineItemsTable details={details} master={master}/> ← 4. Items + totals
|   |   └─ <div class="invoice-bottom-grid"> ← 5. Bottom row (3-col grid)
|   |   |   └─ <PaymentHistory payments={payments} /> (hidden if empty)
|   |   |   └─ <WheelIndicator lf rf lr rr /> (center)
|   |   |   └─ <FinancialSummary master={master} /> (right)
|   |   └─ <InvoiceFooter note companyName /> ← 6. Notes / terms / signature

```

Data contract between parent and child: each section receives exactly the slice of data it needs. `index.tsx` passes either the whole `master` (the `InvoiceMasterDto`) or just the fields a section needs. Most sections take the whole `master` — so adding a new field to a section rarely requires touching `index.tsx`.

Where the report's HTML/JSX is actually written:

The report markup is **JSX inside these `.tsx` files** (Next.js compiles JSX into HTML). There is no separate `.html` file for the report design — the design lives directly in the React components:

What part of the HTML you're editing	File to open
The overall report skeleton — the <code><div ref={printRef}></code> page that stacks all sections in order	<code>index.tsx</code>
Company header block	<code>InvoiceHeader.tsx</code>
Invoice # / date / status row	<code>InvoiceMeta.tsx</code>
Customer + vehicle info boxes	<code>CustomerInfo.tsx</code>
The line-items table (every <code><table></code> / <code><th></code> / <code><td></code> column)	<code>LineItemsTable.tsx</code>
Payment history table	<code>PaymentHistory.tsx</code>
Sidebar financial summary	<code>FinancialSummary.tsx</code>
Wheel indicator SVG + badges	<code>WheelIndicator.tsx</code>
Notes / terms / signature block	<code>InvoiceFooter.tsx</code>

Rule of thumb: open `index.tsx` first — it shows the whole report structure at a glance and tells you which child file owns each section. Then open that section's file to edit its specific HTML. To **add a whole new section**, create a new `.tsx` file in the `invoice-print/` folder and insert its component inside the `ref={printRef}` div in `index.tsx`.

4. ★ The "which file do I edit?" quick reference

This is the table to keep. For every change, **one** file (plus possibly `print.css`).

I want to change...	File to edit	Where in the file
Company name / address / phone shown in header	<code>InvoiceHeader.tsx</code>	the <code><h1></code> / <code><p></code> tags
Add a company logo	<code>InvoiceHeader.tsx</code>	the <code>LOGO_PLACEHOLDER</code> comment
Header accent bar color	<code>InvoiceHeader.tsx</code>	<code>bg-gradient-to-r from-blue-600 to-blue-400</code>
Invoice # / Date / Status shown	<code>InvoiceMeta.tsx</code>	the three flex rows
Status badge colors (Full/Partial/Pending)	<code>InvoiceMeta.tsx</code>	the <code>bg-emerald/amber/red</code> ternary
Customer fields shown	<code>CustomerInfo.tsx</code>	left box's <code><InfoRow ...></code> lines
Vehicle fields shown	<code>CustomerInfo.tsx</code>	right box's <code><InfoRow ...></code> lines
Add / delete / rename a column in the items table	<code>LineItemsTable.tsx</code>	<code><thead></code> + each <code><td></code> + the width style
The item "Description" text (what gets joined)	<code>LineItemsTable.tsx</code>	the <code>itemDescription()</code> helper
The totals under the table (SubTotal/Tax/.../Total)	<code>LineItemsTable.tsx</code>	the "Financial Summary (right-aligned)" block
The sidebar summary (Amount Paid / Balance Due)	<code>FinancialSummary.tsx</code>	the <code><Row ...></code> lines
Payment history columns	<code>PaymentHistory.tsx</code>	<code><thead></code> + <code><td></code> lines
Wheel badge colors / labels	<code>WheelIndicator.tsx</code>	<code>WheelBadge</code> component
Notes / Terms & Conditions / Signature	<code>InvoiceFooter.tsx</code>	the <code>DEFAULT_TERMS</code> string + JSX
Page size (Letter/A4) & margins	<code>print.css</code>	the <code>@page</code> rule
Font size / spacing of the printed report	<code>print.css</code>	the <code>.invoice-print-root</code> font-size rules
Number of columns in the bottom row	<code>index.tsx</code> + <code>print.css</code>	<code>invoice-bottom-grid</code> classes
Add a whole new section to the report	new file + <code>index.tsx</code>	see recipe H in §6
The "Yes, View & Print" prompt after save	<code>invoice-form/index.tsx</code>	the <code>Post-save success dialog</code> block
Where the "View / Print" menu item appears	<code>invoice-datatable/index.tsx</code>	the dropdown around line 350

5. How the design fits together (architecture)

A few deliberate decisions that make the report easy to change:

1. **One folder per concern.** Every visual section is its own tiny component in `invoice-print/` . Changing a section never means digging through one giant file.
2. **`index.tsx` is the only "brain".** It fetches, destructures the response into `master` / `details` / `payments` , and hands each section its slice. Sections are otherwise dumb renderers — they take props and return JSX.
3. **`master` is passed wholesale to most sections.** So a new field in `CustomerInfo.tsx` needs no change to `index.tsx` .

4. **The whole report is one** `<div ref={printRef}>`. That single node is what print CSS targets and what the PDF capture clones. Keep every section inside it.
5. **The line-items table is a raw HTML** `<table>`, not the shadcn Table — print CSS behaves more predictably with native tables, and `table-layout: fixed` keeps column widths stable across screen, print, and PDF.
6. **Two financial summaries exist, deliberately:**
 - `LineItemsTable.tsx` shows totals **computed from the line items** (so they always match the rows above).
 - `FinancialSummary.tsx` shows totals **read from the master record** (including Amount Paid / Balance Due).

If you edit one, remember the other still displays its own numbers.

6. Recipes — common modifications

A. Add / delete / rename a column in the items table (most common)

File: `LineItemsTable.tsx`

The table uses `tableLayout: 'fixed'` with explicit **percentage widths per column**. Two things must stay in sync:

1. The `<th>` header cell (with its `style={{ width: 'X%' }}`).
2. The matching `<td>` data cell in each row.

All widths **must add up to 100%** (currently `4 + 6 + 42 + 16 + 10 + 11 + 11 = 100`).

Example — delete the "Bolt" column:

```
// 1) Remove the header:
<th className='px-1.5 py-1.5 text-left ...' style={{ width: '10%' }}>
  Bolt
</th>

// 2) Remove the data cell:
<td className='px-1.5 py-1.5 text-slate-600 truncate' title={item.tbid_Bolt || '-'}>
  {item.tbid_Bolt || '-'}
</td>

// 3) Give the 10% back to another column, e.g. Description 42% → 52%.
```

Example — add a "Brand" column (data exists on the item as `tbid_Brand`):

```
// 1) Add a header. Take width from Description (e.g. 42% → 34%):
<th className='px-1.5 py-1.5 text-left font-semibold text-slate-600 uppercase tracking-wider' style={{ width: '8%' }}>
  Brand
</th>

// 2) Add the data cell in the same position in each <tr>:
<td className='px-1.5 py-1.5 text-slate-600 truncate' title={item.tbid_Brand || '-'}>
  {item.tbid_Brand || '-'}
</td>

// 3) If the column is wide, note that the Description is also in `itemDescription()` –
// you may want to remove it from there too (see recipe B).
```

If the total goes above 100%, columns get squeezed and text wraps — keep the sum at exactly 100.

B. Change what the "Item Description" column shows

File: `LineItemsTable.tsx`, the `itemDescription()` helper at the top:

```
const itemDescription = (d: InvoiceDetailsDto) =>
[
  d.tbid_DepartmentName,
  d.tbid_Size,
  d.tbid_Brand,
  d.tbid_Series,
  d.tbid_Bolt,
  d.tbid_HoleS,
  d.tbid_Zone,
]
.filter((v) => v !== null && v !== undefined && String(v).trim() !== '')
.join(', ')
```

Add or remove any field of the detail DTO in the array. Empty values are filtered out automatically. (You'll find the available field names in the types file — see §7.)

C. Add / remove a field in Customer or Vehicle info

File: `CustomerInfo.tsx`

Each field is one `<InfoRow label='...' value={master....} />`. Fields with empty values render nothing automatically.

```
// Add a field (e.g. a work phone):
<InfoRow label='Work Phone' value={master.tbim_Phone} />

// Remove a field: just delete its <InfoRow> line.
```

`InfoRow` is defined at the bottom of the same file. If the field doesn't exist on the DTO yet, add it to the type first (see §7).

D. Change the company header / add a logo

File: `InvoiceHeader.tsx`

The header shows `master.tbim_CompanyName`, `master.tbim_CompanyAddress`, and `master.tbim_Phone`, with sensible fallbacks. To add a logo, replace the `LOGO_PLACEHOLDER` comment:

```

```

E. Change colors / branding

Every color is a Tailwind class on one of the section files. The main brand color is **blue-600**.

Element	Class to change
Header accent bar	<code>from-blue-600 to-blue-400</code> in <code>InvoiceHeader.tsx</code>
Selected wheel badge	<code>bg-blue-600</code> in <code>WheelIndicator.tsx</code>
Tax checkbox tick	<code>border-blue-500 bg-blue-500</code> in <code>LineItemsTable.tsx</code>
Payment status badge	<code>bg-emerald-50/amber-50/red-50</code> in <code>InvoiceMeta.tsx</code>
Section title labels	<code>text-slate-400</code> / <code>text-slate-500</code> across all files

For a global look, search `invoice-print/` for `blue-` and replace. (Remember: SVG fills in `WheelIndicator.tsx` use **hex colors**, not Tailwind classes — see gotcha §10.2.)

F. Change the Terms & Conditions

File: `InvoiceFooter.tsx` — edit the `DEFAULT_TERMS` constant:

```
const DEFAULT_TERMS =  
  "All sales are final. It is the customer's responsibility to recheck ..."
```

G. Change page size, margins, fonts

File: `print.css`

```
@page {  
  size: letter portrait;      /* change to e.g. 'A4 portrait' */  
  margin: 0.3in 0.3in;       /* page margins */  
}
```

Font sizes for print are set near the bottom of the file:

```
.invoice-print-root { font-size: 11px !important; line-height: 1.4 !important; }
```

Important: the PDF download path captures the invoice at a **fixed 816 px width** (= 8.5in US Letter @ 96 DPI) set in `index.tsx` (`PAGE_WIDTH_PX`). If you change the page to A4, that number must become **794** (A4 @ 96 DPI). See §8.

H. Add a whole new section to the report

- 1. Create a new file in `invoice-print/` (e.g. `WarrantyNote.tsx`) that takes the props it needs and returns JSX. Follow the styling of the existing sections (`border border-slate-200 rounded-lg p-3` cards, `text-[10px] uppercase` titles).
- 2. Import it in `index.tsx`.
- 3. Add it inside the `ref={printRef}` div, between the existing sections.
- 4. If it's part of the bottom row, put it in the `invoice-bottom-grid` div and adjust that grid in `index.tsx` and `print.css`.
- 5. Give it the data it needs — either `master` (already destructured) or a new destructure from `data`.

I. Show the prompt after save (or change its text/buttons)

File: `invoice-form/index.tsx` — search for `Post-save success dialog`. The dialog text, the two buttons, and the navigation targets are all right there.

7. Where the data comes from (field names)

Types file: `src/app/(DashboardLayout)/types/apps/invoiceMaster.ts`

The print page receives the list-style response shape `InvoiceListResponseItem`:

```
{
  invoiceMasterDto: InvoiceMasterDto,    // → const master
  invoiceDetailsDto: InvoiceDetailsDto[], // → const details
  invoicePaymentsDto: InvoicePaymentsDto[], // → const payments
}
```

Slice	Key fields you'll most often use
<code>master</code> (<code>InvoiceMasterDto</code>)	<code>id</code> , <code>tbim_InvoiceIdRad</code> (invoice #), <code>tbim_InvDate</code> , <code>tbim_Name</code> , <code>tbim_CompanyName</code> , <code>tbim_CompanyAddress</code> , <code>tbim_Phone</code> , <code>tbim_EmailAddress</code> , <code>tbim_IDNo</code> , <code>taxIdentificationNumber</code> , <code>taxCompanyName</code> , <code>tbim_VehicleMake</code> , <code>tbim_Model</code> , <code>tbim_Year</code> , <code>tbim_Odometer</code> , <code>tbim_TreadDepth</code> , <code>tbim_License</code> , <code>tbim_SubTotal</code> , <code>tbim_SaleTax</code> , <code>tbim_Labour</code> , <code>tbim_DisPer</code> , <code>tbim_DisAmt</code> , <code>tbim_Total</code> , <code>tbim_AdjAmt</code> , <code>tbim_AdjTotal</code> , <code>tbim_PaidAmt</code> , <code>tbim_PayInfo</code> , <code>tbim_Note</code> , <code>tbim_Left_Front</code> , <code>tbim_Right_Front</code> , <code>tbim_Left_Rear</code> , <code>tbim_Right_Rear</code>
<code>details[i]</code> (<code>InvoiceDetailsDto</code>)	<code>tbid_DepartmentName</code> , <code>tbid_Size</code> , <code>tbid_Brand</code> , <code>tbid_Series</code> , <code>tbid_Bolt</code> , <code>tbid_HoleS</code> , <code>tbid_Zone</code> , <code>tbid_Qty</code> , <code>tbid_Taxable</code> , <code>tbid_UnitPrice</code> , <code>tbid_LineTotal</code> , <code>tbid_TaxAmt</code>
<code>payments[i]</code> (<code>InvoicePaymentsDto</code>)	<code>tbip_PayAmt</code> , <code>tbip_Date</code> , <code>tbip_PaymentType</code> , <code>paymentName</code>

To show a field that isn't rendered yet: find it in the type above, then add an `<InfoRow>` / column / `<Row>` referencing `master.<field>` or `item.<field>`. If the backend doesn't return it at all, the type must be extended (add the field to the interface) — that's the only backend-adjacent change.

8. How PDF generation works (the important part)

File: `src/app/components/react-tables/transaction/invoice-print/index.tsx` — the `handleDownloadPdf()` function.

Libraries: `html-to-image` (DOM → PNG) + `jsPDF` (PNG → PDF). Both were already installed.

The flow:

1. Take the report node (`printRef.current`).
2. **Clone it** into an off-screen wrapper fixed at `816px` wide (US Letter @ 96 DPI):

```
const wrapper = document.createElement('div')
wrapper.style.width = '816px'
document.body.appendChild(wrapper)
const clone = node.cloneNode(true)
clone.style.width = '816px' // force exact width
wrapper.appendChild(clone)
```

Why the clone? The dashboard's flex layout + scroll containers can make the live node render wider or shifted than the visible report. Capturing the live node gave cut-off pages (right half missing). Cloning into a plain off-screen div gives a predictable 816px box to capture.

1. Render it to a PNG at `pixelRatio: 1`:

```
const dataUrl = await toPng(clone, {
  quality: 1.0,
  pixelRatio: 1, // PNG is exactly 816px wide
  backgroundColor: '#ffffff',
  width: 816,
  height: clone.scrollHeight,
  cacheBust: true,
})
```

1. Feed the PNG into a US-Letter portrait jsPDF and fit it to the page width:

```
const pdf = new jsPDF({ unit: 'in', format: 'letter', orientation: 'portrait' })
pdf.addImage(dataUrl, 'PNG', 0, 0, pageWidth, pdfHeight)
```

If the image is taller than one page, it slices into multiple pages (`while (y < imgProps.height)`).

1. `pdf.save('invoice-{id}.pdf')`.

File name: comes from `data?.invoiceMasterDto?.tbim_InvoiceIdRad || invoiceId`.

If you switch page size (recipe G): `PAGE_WIDTH_PX` must match — **816** for Letter, **794** for A4.

9. How browser printing works

File: `print.css` (imported by `index.tsx`)

- `@page { size: letter portrait; margin: 0.3in 0.3in; }` sets the paper and margins.
- A broad selector hides all dashboard chrome:

```
nav, header, aside, [data-sidebar], [role='navigation'], .sidebar, .header, .customizer, footer {
  display: none !important;
}
```

- `.invoice-print-root` is forced to `width: 100%`, no padding, no shadow.
- Tables get `table-layout: fixed !important` so columns don't re-flow.
- The bottom row collapses to 2 columns in print:

```
.invoice-bottom-grid { grid-template-columns: 1fr 1fr !important; }
```

- Fonts shrink to 11px and `-webkit-print-color-adjust: exact` preserves background colors.

Buttons marked `no-print` (the action bar) are hidden in print.

10. Gotchas & troubleshooting

1. **Keep column widths at exactly 100%.** `LineItemsTable.tsx` uses `tableLayout: 'fixed'`; if widths overflow, text wraps or the last column gets cut off.
2. **Tailwind classes don't work inside inline SVG.** The car in `WheelIndicator.tsx` originally rendered all-black with `fill-slate-100`. Fix: use **explicit hex** (`fill="#f1f5f9"` , `stroke="#cbd5e1"`). If you edit the car SVG, keep hex colors.
3. **Don't wrap the items table in `overflow-x-auto`.** A scroll container lets the table render wider than its parent, which caused the right half of the PDF to be cut off. The table is `w-full` inside `overflow-hidden` instead.
4. **The create-response id is nested.** `CreateInvoice` returns the invoice object directly — id at `result.invoiceMasterDto.id`. The extractor in `invoice-form/index.tsx` handles several shapes and logs a warning if it can't find one. If it breaks again, check the browser console warning for the real shape.
5. **PDF cut-off symptoms = the capture width.** If the PDF ever shows only part of the report again, re-check `PAGE_WIDTH_PX` (816) matches the page size, that `pixelRatio` is `1`, and that the items table has no horizontal-scroll wrapper.
6. **Two summaries.** `LineItemsTable.tsx` computes totals from items; `FinancialSummary.tsx` reads them from the master. They're intentionally both shown. Editing "the summary" means deciding which one — usually the sidebar (`FinancialSummary.tsx`) is the one people mean.
7. **Status badge in `InvoiceMeta.tsx`** keys off `tbim_PayInfo`: `F` = Full (emerald), `P` = Partial (amber), anything else = Pending (red). If the backend sends different codes, update the ternary and the `PAY_LABEL` map.
8. **After editing, hard-refresh / restart the dev server.** Next.js caches modules; a stale bundle can make you think a fix didn't work.

Key design decisions (why it's built this way)

Decision	Why
Separate <code>invoice-print/</code> folder	Each section is independently editable; one concern per file
<code>window.print()</code> instead of <code>react-to-print</code>	Simpler; <code>@media print</code> CSS handles everything reliably
Raw <code><table></code> + <code>table-layout: fixed</code>	Predictable column widths across screen, print, and PDF
Clone-in-wrapper for PDF capture	Fixes the cut-off-right-half problem caused by the dashboard's flex/scroll layout
Grid-based <code>WheelIndicator</code>	More reliable than the form's absolute-positioned <code>CarVisual</code>
The report is one <code>ref</code> node	One element to style for print and clone for PDF
Post-save prompt	So users can immediately view/print without hunting the list

Generated as part of the invoice print feature. Update this document whenever the report design changes.